

Ejercicio Demostrativo

Nombre del Auxiliar

Tiempo estimado: XX hrs/min

Universidad San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ingeniería en Ciencias y Sistemas.
Segundo Semestre 2026
[Laboratorio - Nombre del Curso - Sección]

1. Marco Formativo

Nombre del valor	¿Cómo se aplica en tu laboratorio?
Integridad académica	La integridad académica se evidencia cuando el estudiante escribe su propio código, documenta correctamente sus clases y métodos, y utiliza el control de versiones (Git) para registrar el historial real de su trabajo sin falsificar commits.

1.1 Valores

1.2 Competencias

Tipo de Competencia	Redacción
Competencia General	Desarrollar aplicaciones de software utilizando el paradigma de Programación Orientada a Objetos en C#, aplicando los pilares de la POO y gestionando el ciclo de vida del proyecto mediante herramientas de control de versiones con Git.
Competencia Específica	Modelar clases con atributos y métodos, aplicar herencia, polimorfismo, abstracción y encapsulamiento en C#, e integrar el proyecto con Git utilizando ramas (branches), merging y etiquetas (tags) siguiendo el flujo Git Flow.

2. Palabras Clave

POO (Programación Orientada a Objetos): Paradigma de programación que organiza el código en objetos que combinan estado y comportamiento. Clase: Plantilla o molde que define los atributos y métodos comunes a un grupo de objetos. Herencia: Mecanismo por el cual una clase hija hereda atributos y métodos de una clase padre. Polimorfismo: Capacidad de un objeto de comportarse de distintas formas según el contexto o el tipo con el que se referencia. Git: Sistema distribuido de control de versiones que permite registrar, compartir y revertir cambios en el código fuente. Git Flow: Modelo de ramificación para Git que define ramas principales (main, develop) y ramas de soporte (feature, release, hotfix).

3. Resumen

Este ejercicio demostrativo presenta de forma guiada los fundamentos de la Programación Orientada a Objetos en C# y el control de versiones con Git, cubriendo desde la definición de clases y objetos hasta la implementación de los cuatro pilares de la POO, la introducción a GIT y el flujo de trabajo Git Flow con ramas y etiquetas.

La clase se orienta a que el estudiante modele un sistema orientado a objetos real, identifique los pilares de la POO en el código, inicialice un repositorio Git, cree ramas de trabajo y las integre correctamente mediante merging siguiendo el flujo Git Flow.

4. Contenido

Descripción del ejercicio

Este ejercicio consiste en modelar un sistema de gestión de empleados usando POO en C#: se definen clases base y derivadas, se aplican los cuatro pilares de la POO, y el proyecto se versiona con Git usando el flujo Git Flow (ramas feature, develop, main). Al finalizar, el estudiante tendrá un proyecto C# funcional con jerarquía de clases y un repositorio Git con historial de commits, ramas y etiquetas (tags).

Objetivo de la demostración

Demostrar el procedimiento para modelar clases con herencia y aplicar los pilares de la POO en C#, gestionando el proyecto con Git Flow, con el fin de que el estudiante comprenda cómo estructurar código orientado a objetos y mantener un historial de versiones ordenado.

Recursos necesarios

Recursos utilizados en la demostración:

- Software: Visual Studio Community 2022 o VS Code con extensión C#. .NET SDK 8.0. Git 2.x instalado en el sistema.
- Herramientas técnicas: Compilador C# integrado, depurador de Visual Studio, terminal Git Bash o PowerShell, GitHub o GitLab como repositorio remoto.
- Materiales de apoyo: Diagrama UML de clases del sistema de empleados, guía de comandos Git básicos, referencia del modelo Git Flow.
- Equipo: Computadora con 8 GB de RAM mínimo, conexión a internet para clonar/push a repositorio remoto.

- Configuración: Git configurado con nombre de usuario y correo (git config --global). Cuenta activa en GitHub o GitLab.

Desarrollo paso a paso

Nota: Describe el procedimiento en orden lógico y operativo, recuerda que esto será utilizado por otro tutor ya que tu práctica dura dos semestres y en el siguiente estará otro tutor posiblemente.

Cada paso debe redactarse con claridad, evitando explicaciones innecesarias, pero incluyendo lo suficiente para que se entienda qué se hace, por qué se hace y qué resultado parcial se espera.

Paso 1

Introducción a POO y definición de clases y objetos. Explicar el paradigma POO y crear la clase base Empleado con atributos (nombre, salario) y métodos (calcularBono(), mostrarInfo()). Instanciar objetos y mostrar sus propiedades en consola. El estudiante debe identificar la diferencia entre clase y objeto.

Paso 2

Aplicación de herencia, polimorfismo, abstracción y encapsulamiento. Crear clases derivadas (EmpleadoTiempoCompleto, EmpleadoPorHoras) que hereden de Empleado. Sobrescribir calcularBono() demostrando polimorfismo. Encapsular atributos con modificadores private y propiedades públicas. Definir una clase o interfaz abstracta. Compilar y verificar que el polimorfismo funcione correctamente.

Paso 3

Introducción a Git: inicialización del repositorio y primeros commits. Ejecutar git init en el directorio del proyecto. Agregar los archivos con git add . y realizar el primer commit con git commit -m "Clase base Empleado". Crear rama develop con git checkout -b develop. Verificar el historial con git log --oneline y confirmar que aparece el commit inicial.

Paso 4

Git Flow: creación de ramas, merging y etiquetas (tags). Crear una rama feature/clases-derivadas, agregar los cambios de herencia y polimorfismo, luego hacer merge a develop. Crear una rama release/1.0, hacer merge a main y etiquetar con git tag v1.0. El ejercicio está resuelto cuando git log --all --oneline muestra el historial completo con las ramas y el tag v1.0.

Observaciones importantes

Siempre configurar Git con nombre y correo antes de hacer commits. No hacer commits directamente en main; usar el flujo de ramas. Usar mensajes de commit descriptivos. En C#, respetar los modificadores de acceso: private para atributos, public para propiedades. Evitar herencia profunda (más de 2 niveles) para mantener el código mantenible.

Tabla 1

Resumen de subtemas y elementos demostrados:

Elemento	Descripción
POO: Clases y Objetos	Clase Empleado con atributos y métodos, instanciación y acceso a propiedades
Pilares de la POO	Herencia, polimorfismo (override), encapsulamiento (private/public) y abstracción (clase abstracta)
Git Workflow (Git Flow)	git init, git add, git commit, ramas (feature/develop/main), merge, git tag v1.0 siguiendo Git Flow

Resultado esperado

Al ejecutar correctamente el ejercicio, el estudiante debe tener un proyecto C# con una jerarquía de clases que demuestre los cuatro pilares de la POO. En el repositorio Git debe verse un historial con commits en ramas feature y develop, merge a main y el tag v1.0. El programa debe compilar sin errores y mostrar en consola el comportamiento polimórfico de las clases. El repositorio debe tener al menos 3 commits, 2 ramas y 1 tag visible con git log --all --oneline.

Aplicación práctica

La POO es el paradigma dominante en el desarrollo de software empresarial con C#, Java y Python. Git Flow es el estándar en equipos profesionales para manejar releases y hotfixes. Estos

conceptos se aplican directamente en desarrollo de aplicaciones web con ASP.NET, servicios en Azure y proyectos open-source en GitHub.

5. Aconsejando al siguiente Tutor para la realización de ejemplos

- Dificultades encontradas: Los estudiantes confunden herencia con composición, y frecuentemente olvidan el override al sobrescribir métodos. En Git, los conflictos de merge generan pánico si no se explican con calma.
- Recomendaciones: Usar un diagrama UML visual proyectado durante toda la práctica. Para Git Flow, mostrar el gráfico de ramas en tiempo real con `git log --all --graph --oneline`.
- Mayor dificultad: Comprender polimorfismo en tiempo de ejecución y resolver conflictos de merge en Git.

6. Referencias

1. Microsoft. (2024). Programación orientada a objetos en C#. <https://learn.microsoft.com/es-es/dotnet/csharp/fundamentals/object-oriented/>
2. Chacon, S. & Straub, B. (2014). Pro Git (2da ed.). Apress. <https://git-scm.com/book/es/v2>